

UNIVERSIDADE DO OESTE DE SANTA CATARINA – UNOESC
CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

JOÃO VITOR BERNARDON SCHWEIKART

PRÁTICAS EXTENSIONISTAS IV

DIÁRIO ESCOLAR MUNICIPAL: MODELAGEM ARQUITETURAL DE UMA PWA
PARA ESCOLAS MUNICIPAIS DO INTERIOR

CHAPECÓ – SC, setembro de 2026.

1 INTRODUÇÃO

Este trabalho apresenta a modelagem arquitetural de um sistema voltado a um problema social: a dificuldade de acesso das escolas municipais do interior a sistemas escolares para o lançamento do diário de classe e das notas. O obstáculo não é apenas a ausência de um sistema, e sim a dependência de conexão contínua. A internet dessas escolas costuma ser instável ou inexistente dentro da sala de aula, que é exatamente o lugar e o momento em que o registro deveria acontecer. Na prática, o professor anota em papel e transcreve depois, o que gera retrabalho, atraso e risco de erro em um dado que afeta a vida acadêmica do aluno.

A solução modelada é o Diário Escolar Municipal (DEM), uma PWA (*Progressive Web App*) construída como monolito modular em Laravel. A característica arquitetural central é a operação em rede limitada, onde o professor registra aula, frequência e notas sem conexão, e o sistema sincroniza os lançamentos quando a rede volta a ficar disponível.

O produto desta atividade é a modelagem arquitetural, e não a implementação do software. Seguindo o conteúdo da unidade, a arquitetura é apresentada em três visões complementares: o diagrama de arquitetura da aplicação, expresso em UML como diagrama de pacotes; o diagrama de arquitetura de implantação, que mostra a disposição física dos componentes; e o diagrama de arquitetura DevOps, que mostra o fluxo automatizado do desenvolvimento até a produção. As três visões descrevem uma única arquitetura: os nomes de componentes, serviços e ambientes são os mesmos nos três diagramas.

2 ESCOPO E PREMISSAS DE MODELAGEM

O escopo foi dimensionado para sustentar as decisões arquiteturais sem transformar o trabalho em um projeto de grande porte. O sistema reconhece três papéis: Administrador da Secretaria, que mantém a estrutura da rede municipal; Gestor Escolar, que acompanha o diário da sua unidade; e Professor, único perfil que opera sem conectividade. Estão contemplados a autenticação e a autorização por papel, os cadastros de escolas, professores, turmas, disciplinas, alunos e matrículas, o diário escolar com conteúdo de aula e frequência, as avaliações, o lançamento e a consulta de notas com cálculo de média, a persistência relacional e a sincronização dos lançamentos feitos offline.

Cada instalação atende um município, com várias escolas sob a mesma secretaria. Somente o professor opera offline; gestão e secretaria operam sempre com conexão. O professor acessa apenas as turmas e disciplinas às quais está vinculado. O dimensionamento assumido é de até 30 escolas, 400 professores e 9.000 alunos por município. Os identificadores das entidades criadas offline são IDs gerados no cliente, o que evita colisão de chaves. Conflitos de conteúdo de aula e de frequência são resolvidos por *last-write-wins*, mas conflitos de nota exigem decisão

humana do gestor, por se tratar de dado com efeito acadêmico. Por fim, assumem-se objetivos de recuperação de até 4 horas e de até 24 horas, compatíveis com backup diário.

3 ARQUITETURA DA APLICAÇÃO

A UML representa a arquitetura da aplicação por meio do diagrama de pacotes, isto é, das divisões lógicas do sistema. A Figura 1 descreve como o software está dividido — e não onde ele executa, assunto do capítulo 4.

No lado do cliente, *pwa.ui* reúne as telas Blade e os componentes Alpine.js, e *pwa.offline* concentra o que torna a aplicação utilizável sem rede: o *service worker* construído com Workbox, o cache da interface e dos dados da turma e a fila local de lançamentos gravada no IndexedDB do navegador do usuário.

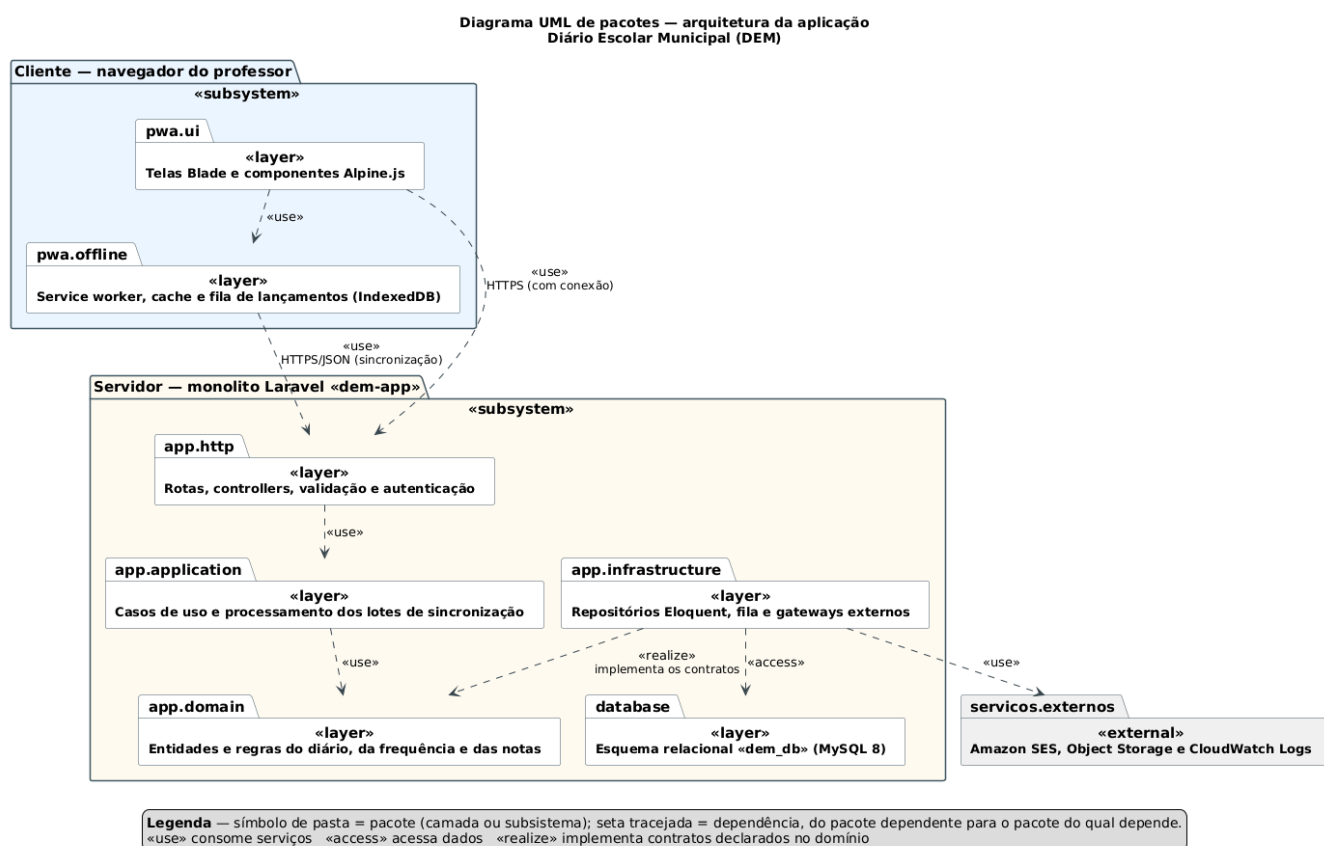
No servidor, o monolito Laravel está dividido em cinco camadas: *app.http* é o ponto de entrada, com rotas, *controllers*, validação e autenticação. *app.application* implementa os casos de uso, como registrar aula, lançar frequência, lançar notas e processar os lotes de sincronização. *app.domain* é o núcleo do sistema, com as entidades, as regras de cálculo de média e de apuração de frequência, as políticas de acesso e os contratos que a infraestrutura precisa implementar. *app.infrastructure* implementa esses contratos por meio de repositórios Eloquent, da fila e dos *gateways* externos. *database* guarda o esquema relacional *dem_db*.

A seta tracejada do diagrama parte sempre do pacote dependente e aponta para o pacote do qual ele depende. O ponto central é que *app.domain* não depende de nenhuma outra camada: a ligação entre os contratos e os repositórios Eloquent é resolvida em tempo de execução pelo container de serviços do Laravel, aplicando a inversão de dependência. Na prática, trocar o banco ou o provedor de e-mail não altera o núcleo de regras do diário.

Os serviços externos aparecem em um único pacote porque só três se justificam no escopo: o Amazon SES, exigido pela recuperação de senha por e-mail; o Lightsail Object Storage, exigido pelo backup diário; e o CloudWatch Logs, que preserva os registros da aplicação quando o container é substituído.

Quanto à sincronização, cada operação feita sem conexão recebe um identificador único e é gravada na fila local junto com a versão do registro que estava em cache. O servidor trata esse identificador como chave única, de modo que um lote reenviado não produz efeito duplicado; e compara a versão informada com a versão corrente para detectar conflitos.

Figura 1 – Diagrama UML de pacotes: arquitetura da aplicação



4 ARQUITETURA DE IMPLANTAÇÃO

O diagrama de implantação mapeia a disposição física dos componentes: em quais nós cada artefato é implantado e por quais protocolos os nós conversam. A Figura 2 representa o ambiente de produção e responde onde cada parte executa.

Em qualquer dispositivo que o professor esteja utilizando para acessar o sistema (notebook, *tablet* ou celular), o navegador hospeda o *app shell* e o *service worker*, correspondentes aos pacotes *pwa.ui* e *pwa.offline*, além do IndexedDB com a fila de lançamentos. É esse conjunto que permite a operação sem conectividade: durante a aula, nenhuma requisição ao servidor é necessária.

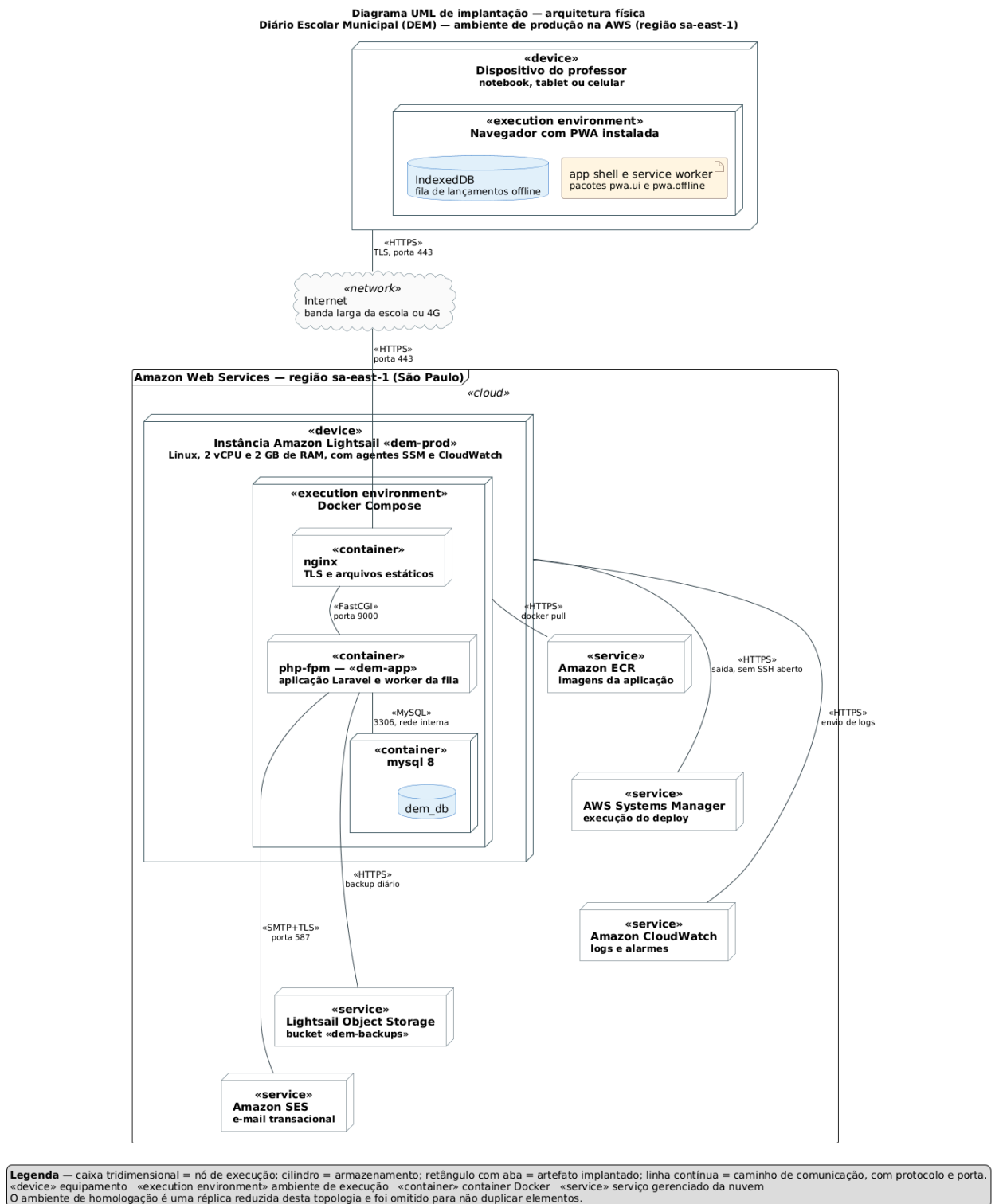
Do outro lado, uma instância Amazon Lightsail com Linux, 2 vCPU e 2 GB de RAM executa três containers sob o Docker Compose: o *nginx*, que termina o TLS e serve os arquivos estáticos; o *php-fpm* com a imagem *dem-app*, que processa as requisições e também executa o *worker* da fila; e o *mysql*, que guarda o banco *dem_db*. A instância mantém instalados os agentes do Systems Manager e do CloudWatch.

Duas decisões de segurança ficam visíveis no diagrama. O banco não é exposto fora da rede interna do Docker, sendo alcançável apenas pela aplicação na porta 3306, aumentando a sua

segurança. E o canal usado pelo *pipeline* para implantar é uma conexão de saída do agente do Systems Manager, o que dispensa manter a porta 22 aberta para a internet.

Uma parada do servidor adia a sincronização e não impede o registro em sala, porque a PWA opera offline. A recuperação se apoia no snapshot diário da instância, no *dump* diário do banco enviado ao *bucket* de backups e na imutabilidade das imagens publicadas no Amazon ECR, que permitem recriar o servidor em outra instância. O ambiente de homologação é uma réplica reduzida da mesma topologia e foi omitido do diagrama para não duplicar elementos idênticos.

Figura 2 – Diagrama UML de implantação: arquitetura física do ambiente de produção



5 ARQUITETURA DEVOPS

O diagrama DevOps não descreve o software nem a infraestrutura: descreve o processo que leva uma alteração de código do computador do desenvolvedor até a produção e traz de volta

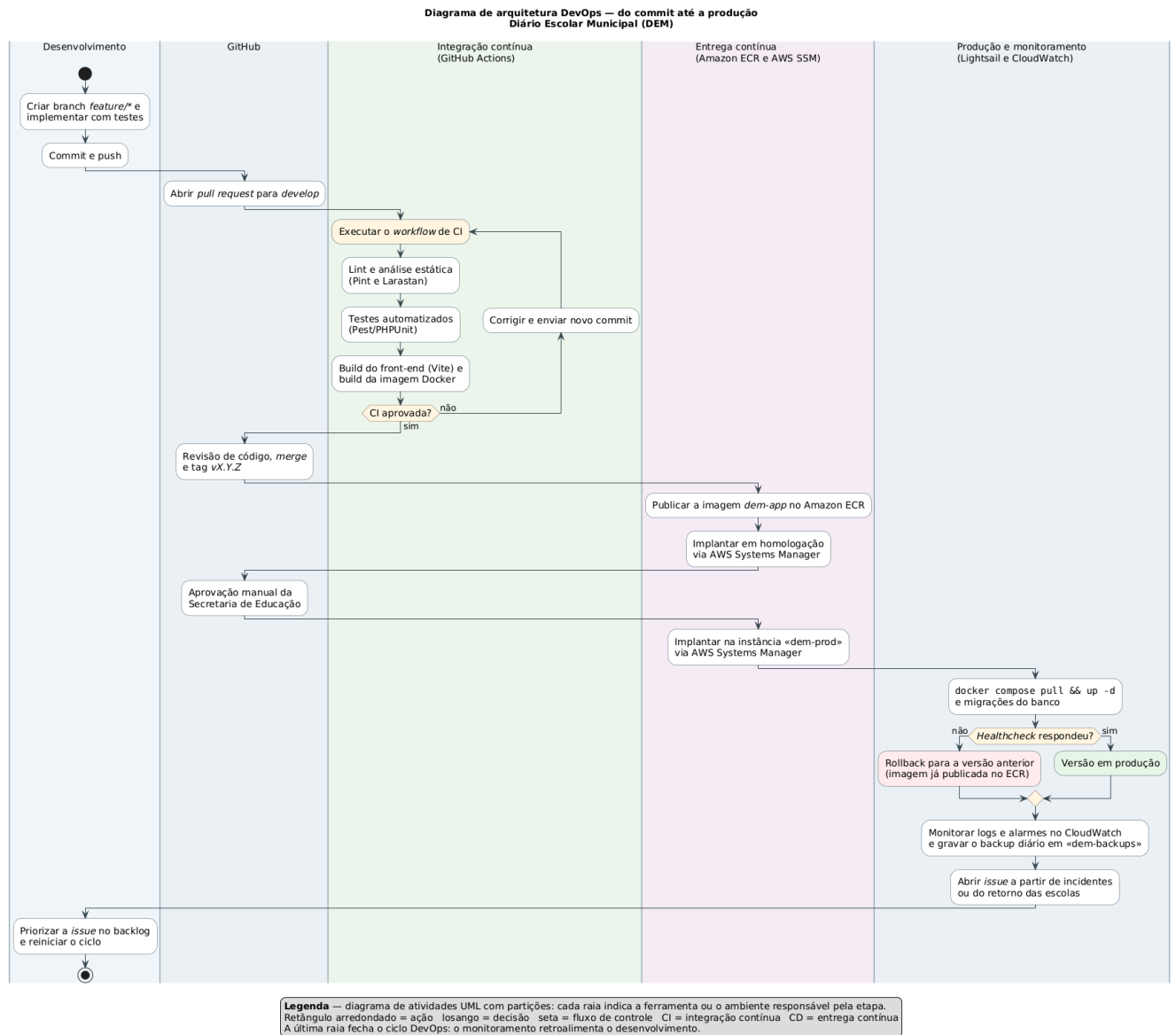
os sinais da operação. Ele foi modelado como diagrama de atividades UML com partições, em que cada raia identifica a ferramenta ou o ambiente responsável pela etapa.

O versionamento segue *Git Flow* simplificado, com *branches feature/** originadas de *develop* e promoção para *main* por meio de *tag* semântica. A cada *pull request*, a integração contínua no GitHub Actions executa *lint* e análise estática com Pint e Larastan, os testes automatizados com Pest/PHPUnit, o *build* do *front-end* com Vite e o *build* da imagem Docker. Se alguma etapa reprova, o *merge* é bloqueado e o ciclo retorna à *branch* de origem — retorno representado no diagrama como um laço, e não como um fim de fluxo.

Aprovada a CI e feito o *merge*, a entrega contínua publica a imagem no Amazon ECR e implanta em homologação por meio do AWS Systems Manager, sem chaves de acesso estáticas no repositório. A promoção para produção exige aprovação manual da Secretaria de Educação, porque o sistema guarda registro acadêmico oficial e a decisão de publicar não deve ser puramente técnica. Em produção, a mesma imagem já testada é implantada e, se o *healthcheck* falhar, o *deploy* retorna à versão anterior: como a imagem antiga permanece no ECR, o *rollback* é uma troca de *tag* e não exige novo *build*.

Por fim, o CloudWatch recebe logs e métricas e dispara alarmes, o backup diário é executado e os incidentes ou retornos das escolas viram *issues* priorizadas no *backlog*. É essa última raia que fecha o ciclo DevOps, fazendo o diagrama retornar à primeira raia em vez de terminar em um ponto isolado.

Figura 3 – Diagrama de arquitetura DevOps: do commit até a produção



6 INFRAESTRUTURA DE PUBLICAÇÃO E JUSTIFICATIVA

A publicação ocorre na Amazon Web Services, região sa-east-1 (São Paulo), escolhida pela proximidade, que reduz a latência na sincronização, e por manter dados pessoais de alunos menores de idade em território nacional. A computação fica em uma instância Amazon Lightsail com Docker Compose; o apoio vem do Amazon ECR, do AWS Systems Manager, do CloudWatch, do Amazon SES e do Lightsail Object Storage, além do DNS e dos snapshots automáticos do próprio Lightsail.

Para uma secretaria municipal de pequeno porte, o fator decisivo não é desempenho de pico, e sim previsibilidade de custo e baixo esforço de operação. O Lightsail atende aos dois: tem preço fixo mensal publicado, inclui franquia de transferência, DNS e *firewall* no mesmo console

e é cobrado por hora até o teto do plano. Um diário escolar tem carga concentrada em poucos horários e volume modesto, o que não justifica a complexidade de VPC e serviços gerenciados separados do par EC2 + RDS, nem a elasticidade do Fargate. A crítica legítima à limitação de escala e ausência de alta disponibilidade nativa é atenuada por dois fatos: a AWS documenta a exportação de recursos do Lightsail para o Amazon EC2, o que oferece caminho de migração sem reescrita, e a indisponibilidade do servidor não paralisa a sala de aula.

O banco roda em container na própria instância, e não em serviço gerenciado. A favor: custo adicional zero, latência desprezível e ambiente local idêntico ao de produção. Contra: risco concentrado em um nó e backup sob responsabilidade da equipe. A alternativa gerenciada do Lightsail parte de US\$ 15 por mês, mais que o dobro do custo total previsto, sem ganho perceptível para o volume estimado.

6.1 Utilizar gratuidade da AWS

A recomendação é usar o plano gratuito apenas na fase acadêmica de validação e migrar a conta para o plano pago antes de inserir dados reais de alunos, configurando também um alarme de orçamento no AWS Budgets. O provisionamento segue a ordem usual: criar a conta com e-mail institucional, proteger a conta raiz com MFA, criar as identidades de trabalho no IAM, definir o alarme de orçamento, provisionar a instância e o *bucket*, registrar a instância no Systems Manager e criar o repositório no ECR com autenticação por OIDC.

Por fim, como o sistema trata dados de crianças e adolescentes, a arquitetura prevê TLS em todo o tráfego, banco sem exposição pública, segregação de acesso por papel e por vínculo com a turma, registro de auditoria das alterações de notas e retenção definida para os backups. O cache local no dispositivo do professor é um ponto de atenção explícito: ele deve guardar apenas as turmas do próprio professor e ser limpo no encerramento da sessão.

7 CONSIDERAÇÕES FINAIS

A modelagem parte de uma constatação simples: em escolas municipais do interior, o problema do diário escolar não se resolve apenas com um sistema, mas com um sistema que funcione quando a internet não funciona. Essa exigência organiza todo o restante — a camada de suporte offline no cliente, a sincronização em lotes idempotentes e até a tolerância a um servidor de nó único, aceitável justamente porque a sala de aula não depende dele.

As três visões descrevem uma única arquitetura. O diagrama de pacotes mostra um monolito modular em que o domínio não depende de nenhuma outra camada; o de implantação mostra onde esses pacotes executam; e o DevOps mostra como o código percorre o caminho entre os dois e como a operação retroalimenta o desenvolvimento. Os mesmos nomes aparecem

nos três, o que é a forma mais concreta de verificar que não foram modeladas três arquiteturas diferentes.

Quanto à infraestrutura, a verificação das condições atuais da AWS produziu o resultado mais útil do ponto de vista prático: o Lightsail não é gratuito e, ao fim do plano gratuito, a conta é encerrada. Um trabalho que afirmasse gratuidade sem essa checagem levaria uma secretaria municipal a uma decisão equivocada. O custo real, de cerca de US\$ 15 por mês, é modesto, mas precisa ser previsto no orçamento.